TRƯỜNG ĐẠI HỌC VĂN LANG · KHOA CÔNG NGHỆ THÔNG TIN

BUỔI 11 · LÝ THUYẾT

Tích hợp hệ thống và Xử lý lỗi

Quản lý phụ thuộc · Debug có hệ thống · Logging và xử lý lỗi xuyên lớp · Docker cơ bản
Buổi lý thuyết cuối trước hạn nộp Bài tập 2

Giảng viên chính HK261: **TS. Nguyễn Trí Hải / ThS. Nguyễn Minh Tân / ThS. Trần Công Thanh**

Học kỳ 1, năm học 2026 – 2027

NỘI DUNG BUỔI HỌC

Năm phần, buổi lý thuyết cuối của Giai đoạn 2

01

Quản lý phụ thuộc và khoá phiên bản

Vì sao “trên máy em chạy được” không phải bằng chứng; khoá phiên bản theo từng stack; thêm thư viện có cân nhắc.

02

Debug có hệ thống: bốn bước

Tái lập → khoanh vùng → sửa nguyên nhân → xác nhận bằng test. Kỹ thuật chia đôi; sửa triệu chứng và sửa nguyên nhân; Bug Log.

03

Logging và xử lý lỗi xuyên lớp

Bốn mức log; ba thứ mọi dòng log phải có; thứ tuyệt đối không ghi; lỗi đi qua repository → service → controller thế nào.

04

Docker cơ bản – ở mức chạy được

Image, container, volume, mạng; Dockerfile tối thiểu; thứ tự lớp và cache; docker-compose cho ứng dụng kèm CSDL; lỗi hay gặp.

05

Phần chuyên ngành – Sinh viên tự học theo tài liệu

Đóng gói và cấu hình theo stack [SE] · nối pipeline và chạy theo lịch [DA] · đóng gói mô hình thành dịch vụ dự đoán [AI].

MỤC TIÊU

Mục tiêu buổi học

1 Tái lập được môi trường

- Giải thích ba nguồn khác biệt giữa hai máy
- Khoá phiên bản bằng tệp khoá của stack mình và commit nó

2 Gỡ lỗi có phương pháp

- Áp dụng đủ bốn bước, đúng thứ tự
- Phân biệt sửa triệu chứng với sửa nguyên nhân
- Ghi một mục Bug Log 7 dòng

3 Log và lỗi đúng chỗ

- Chọn đúng mức log; log có thời điểm, ngữ cảnh, mô tả
- Không ghi dữ liệu nhạy cảm
- Dịch lỗi ở ranh giới, ghi log một lần

4 Chạy được bằng Docker

- Phân biệt image, container, volume
- Viết Dockerfile tối thiểu đúng thứ tự lớp
- Dựng app + CSDL bằng docker compose

Bốn mục tiêu này là đầu vào trực tiếp cho thành phần 4 của Bài tập 2 (hệ thống tích hợp chạy được) và cho Bug Log, gói đóng gói của Bài tập 3.

KHÁI NIỆM CỐT LÕI CỦA BUỔI HỌC

Lỗi tích hợp hiếm khi nằm trong một module. Nó nằm ở chỗ các module gặp nhau — và ở chỗ máy em gặp máy người khác.

- Mỗi module đã chạy đúng một mình (buổi 7–10). Khi ghép lại, lỗi xuất hiện ở **ranh giới**: hợp đồng dữ liệu lệch, cấu hình sai, thứ tự khởi động sai.
- Khi chuyển sang máy khác, lỗi xuất hiện ở **môi trường**: phiên bản thư viện, runtime, biến môi trường, dữ liệu cục bộ.
- Hôm nay học bốn công cụ cho hai loại lỗi này: **khoá phiên bản** (để môi trường giống nhau), **debug bốn bước** (để tìm lỗi có phương pháp), **log và xử lý lỗi** (để nhìn thấy lỗi), **Docker** (để đóng gói cả môi trường).
- Tất cả quy về nguyên tắc **TÁI LẬP ĐƯỢC** của học phần: người khác làm đúng README thì phải ra đúng kết quả của em.

01 Quản lý phụ thuộc và khoá phiên bản

“Trên máy em chạy được” · tệp khoá theo stack · thêm thư viện có cân nhắc

Ba nguồn khác biệt giữa máy em và máy người khác

1

Phiên bản thư viện khác nhau

Em cài pandas tháng trước, người chấm cài hôm nay và nhận bản mới hơn — một hàm đã đổi hành vi. Đây là nguồn phổ biến nhất và **khó phát hiện nhất**.

2

Phiên bản runtime khác nhau

Máy em Node 20, máy thầy Node 18: cú pháp mới báo SyntaxError. Python 3.12 so với 3.9: kiểu **str | None** không chạy. Ghi phiên bản runtime trong README là bắt buộc.

3

Biến môi trường và dữ liệu cục bộ

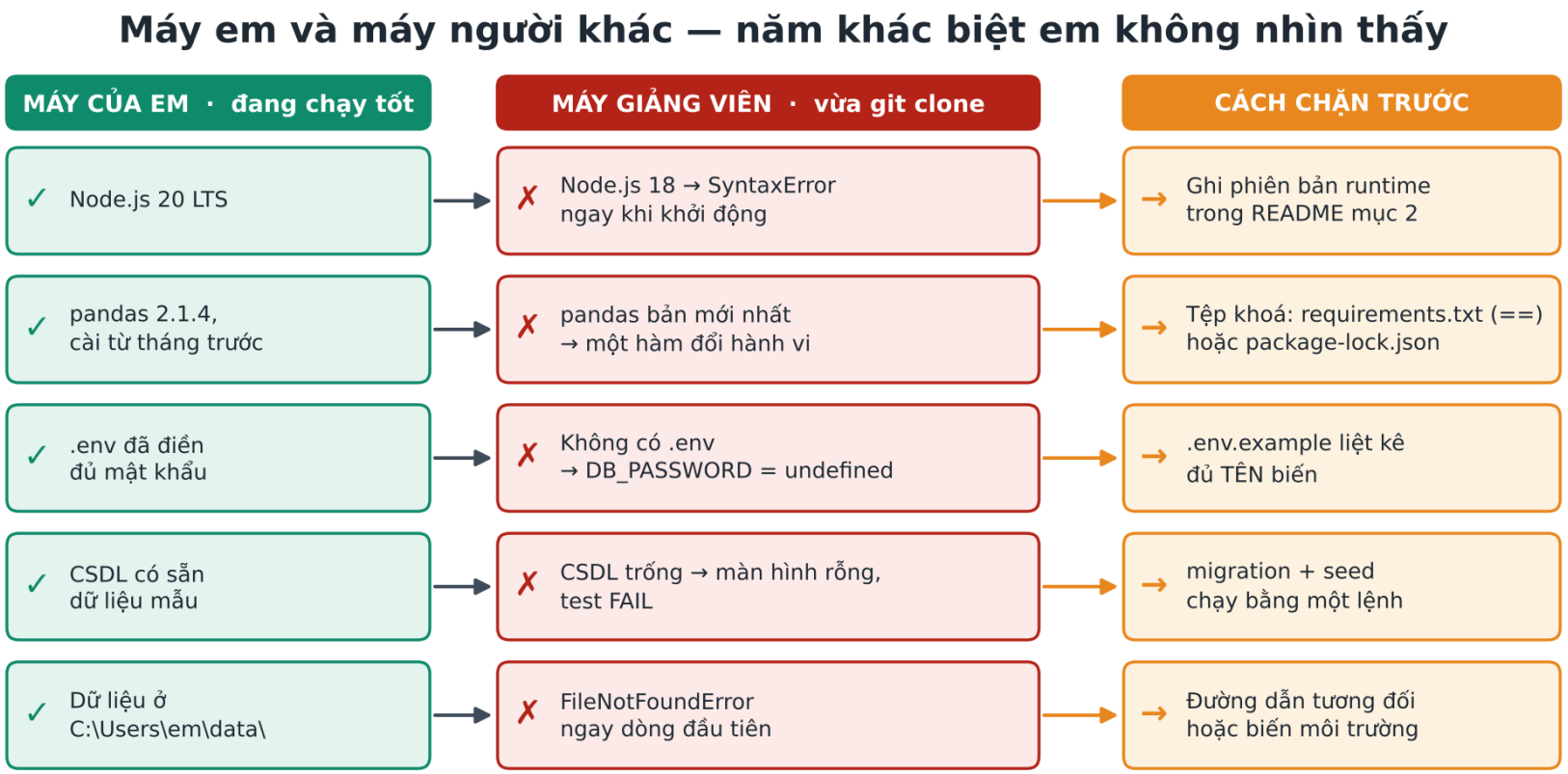
Máy em có .env đã điền, CSDL đã nạp dữ liệu mẫu, tệp nằm ở C:\Users\em\data. Máy người khác **không có gì cả** — vì thế phải có .env.example, migration + seed và đường dẫn tương đối.

Ba việc phải làm trước khi nộp BT2

- Commit tệp khoá (package-lock.json / requirements.txt có ==) — mục C6 của bảng tiêu chí BT2.
- Ghi phiên bản runtime trong README mục 2: “Node.js 20 LTS”, “Python 3.11+”, “JDK 21”.
- Có .env.example đủ TÊN biến, và lệnh nạp dữ liệu mẫu nếu hệ thống cần dữ liệu để chạy.

MINH HOẠ — HAI MÁY, NĂM KHÁC BIỆT

Thứ em không nhìn thấy trên máy mình



Đọc hình thế nào

- Cột trái: mọi thứ "tự nhiên có" trên máy em vì em đã cài, đã điền, đã nạp từ trước.
- Cột giữa: cùng thứ đó trên máy vừa clone — mỗi dòng là một lỗi khác nhau.
- Cột phải: cách chặn trước. Mỗi cách là MỘT tệp hoặc MỘT dòng trong README.
- Phép thử duy nhất phát hiện được cả năm: clone vào thư mục trống và làm đúng theo README.

KHOÁ PHIÊN BẢN THEO STACK

Tệp khai báo, tệp khoá và lệnh cài đúng phiên bản

Stack	Tệp khai báo (em sửa)	Tệp khoá (công cụ sinh ra)	Lệnh cài ĐÚNG phiên bản đã khoá	Lỗi hay gặp
Node.js (npm)	package.json — có thể ghi ^4.18.2	package-lock.json — ghi chính xác 4.18.2 và mọi phụ thuộc con	npm ci	Không commit package-lock.json; dùng npm install khi chấm nên nhận bản mới
Python (pip)	requirements.in hoặc danh sách gói	requirements.txt ghi ==: pandas==2.1.4	pip install -r requirements.txt	Viết tay “pandas, numpy” không có phiên bản; pip freeze từ môi trường bẩn kéo theo gói rác
Python (Poetry)	pyproject.toml	poetry.lock	poetry install --no-root	Sửa pyproject.toml mà quên poetry lock
Java (Maven)	pom.xml	pom.xml ghi phiên bản cụ thể (không dùng LATEST, RELEASE)	mvn -B verify	Dùng dải phiên bản [1.0,) khiến mỗi lần build một khác
Runtime	README mục 2; .nvmrc; .python-version	—	nvm use · pyenv local	README chỉ ghi “cần Node” mà không ghi phiên bản

Quy tắc: tệp khai báo nói “em muốn khoảng nào”, tệp khoá nói “em đã thực sự chạy bản nào”. Người chấm cần tệp khoá. Python: tạo venv MỚI, cài đúng những gì cần, rồi mới pip freeze > requirements.txt.

VÍ DỤ — TỆP KHOÁ TỐT VÀ TỆP KHOÁ TỒI

requirements.txt và package.json: đọc để biết người chấm sẽ cài gì

Cùng một dự án, hai cách ghi phụ thuộc

```
# ===== requirements.txt — TỒI: viết tay, không phiên bản =====
pandas
scikit-learn
sqlalchemy
# → người chấm nhận bản MỚI NHẤT tại thời điểm chấm, có thể khác bản em dùng

# ===== requirements.txt — TỐT: ghim chính xác bằng == =====
pandas==2.1.4
scikit-learn==1.5.2          # mô hình lưu bằng 1.5.2 phải nạp bằng 1.5.2
SQLAlchemy==2.0.36
psycopg2-binary==2.9.10

# Cách tạo đúng: venv MỚI → cài → kiểm chạy được → freeze
python -m venv .venv && source .venv/bin/activate     # Windows: .venv\Scripts\activate
pip install pandas==2.1.4 scikit-learn==1.5.2 SQLAlchemy psycopg2-binary
pip freeze > requirements.txt

// ===== package.json — khai báo KHOẢNG cho phép =====
"dependencies": { "express": "^4.18.2", "pg": "^8.11.3" }
// ^4.18.2 = mọi bản 4.x.y ≥ 4.18.2 → hôm nay 4.18.2, tháng sau có thể 4.21.0
// package-lock.json mới ghi bản THỰC SỰ đã cài → BẮT BUỘC commit
// README và Dockerfile dùng:  npm ci   (cài đúng theo lock)
```

Ghi nhớ

- Tệp khoá là thứ người chấm cần; tệp khai báo chỉ là ý định.
- **npm ci** cài đúng theo lock; **npm install** có thể nâng phiên bản.
- Với track AI: phiên bản scikit-learn lúc lưu mô hình và lúc nạp mô hình phải trùng — lệch là nạp lỗi hoặc dự đoán sai.
- Không commit thư mục .venv hay node_modules — commit TỆP KHOÁ, người khác tự cài.

THÊM THƯ VIỆN CÓ CÂN NHẮC

Bốn câu hỏi trước khi gõ npm install hoặc pip install

1

Em có thật sự cần nó không?

Định dạng một ngày tháng không cần cả một thư viện 300 KB. Mỗi thư viện thêm vào là một thứ có thể lệch phiên bản, có lỗ hổng, và người chấm phải cài.

2

Nó còn được bảo trì không?

Xem lần phát hành gần nhất, số issue đang mở, số người dùng. Thư viện bỏ rơi 3 năm là rủi ro: không tương thích runtime mới, không vá lỗ hổng.

3

Nó kéo theo bao nhiêu thứ khác?

Một gói có thể kéo theo hàng trăm phụ thuộc con. Kiểm bằng **npm ls --all | wc -l** hoặc **pipdeptree**. Ảnh Docker sẽ phình theo.

4

Giấy phép có cho phép dùng không?

MIT, Apache-2.0, BSD: dùng thoải mái. GPL: có ràng buộc khi phân phối. Ghi lại lựa chọn quan trọng vào **docs/thay-doi-thiet-ke.md** nếu nó khác thiết kế BT1.

Thói quen tốt

- Mỗi lần thêm thư viện là MỘT commit riêng: “chore(deps): thêm zod 3.23 để kiểm tra đầu vào”.
- Gỡ thư viện không còn dùng: npm uninstall / pip uninstall rồi cập nhật tệp khoá.
- Thư viện do AI gợi ý: kiểm tên gói có thật không trước khi cài — tên gói “gần giống” là một kiểu tấn công chuỗi cung ứng.
- Khai báo trong ai-disclosure.md nếu AI gợi ý thư viện đó.

02

Debug có hệ thống

Tái lập → khoanh vùng → sửa nguyên nhân → xác nhận · kỹ năng được hỏi ở mọi buổi workshop và ở câu 3 đề thi

BỐN BƯỚC – LÀM ĐÚNG THỨ TỰ, KHÔNG NHẢY BƯỚC

Quy trình gỡ lỗi có hệ thống

1

1. TÁI LẬP (reproduce)

Tìm bộ đầu vào nhỏ nhất làm lỗi xuất hiện MỖI LẦN chạy.

Chưa tái lập được thì chưa được sửa gì — em sẽ không biết mình có sửa đúng không.

→

2

2. KHOANH VÙNG (isolate)

Chia đôi đường đi của dữ liệu, kiểm ở giữa: dữ liệu còn đúng ở đây không?

Lặp lại tới khi còn đoạn mã ngắn nhất chứa lỗi.

→

3

3. SỬA (fix)

Sửa NGUYÊN NHÂN, không sửa triệu chứng.

Thêm một câu if để né lỗi là sửa triệu chứng — lỗi sẽ quay lại ở chỗ khác.

→

4

4. XÁC NHẬN (verify)

Chạy lại bộ đầu vào ở bước 1: lỗi phải biến mất.

Chạy lại toàn bộ test: không vỡ chỗ khác.

VIẾT MỘT TEST cho lỗi này.

Bước 4 sinh ra một **unit test mới** cho mỗi lỗi em sửa — cách tự nhiên nhất để đủ số test BT2 yêu cầu (C4). Ở BT3, Bug Log yêu cầu **≥ 3 bug kèm link commit sửa**: mỗi lần chạy đủ bốn bước là em có sẵn một mục Bug Log.

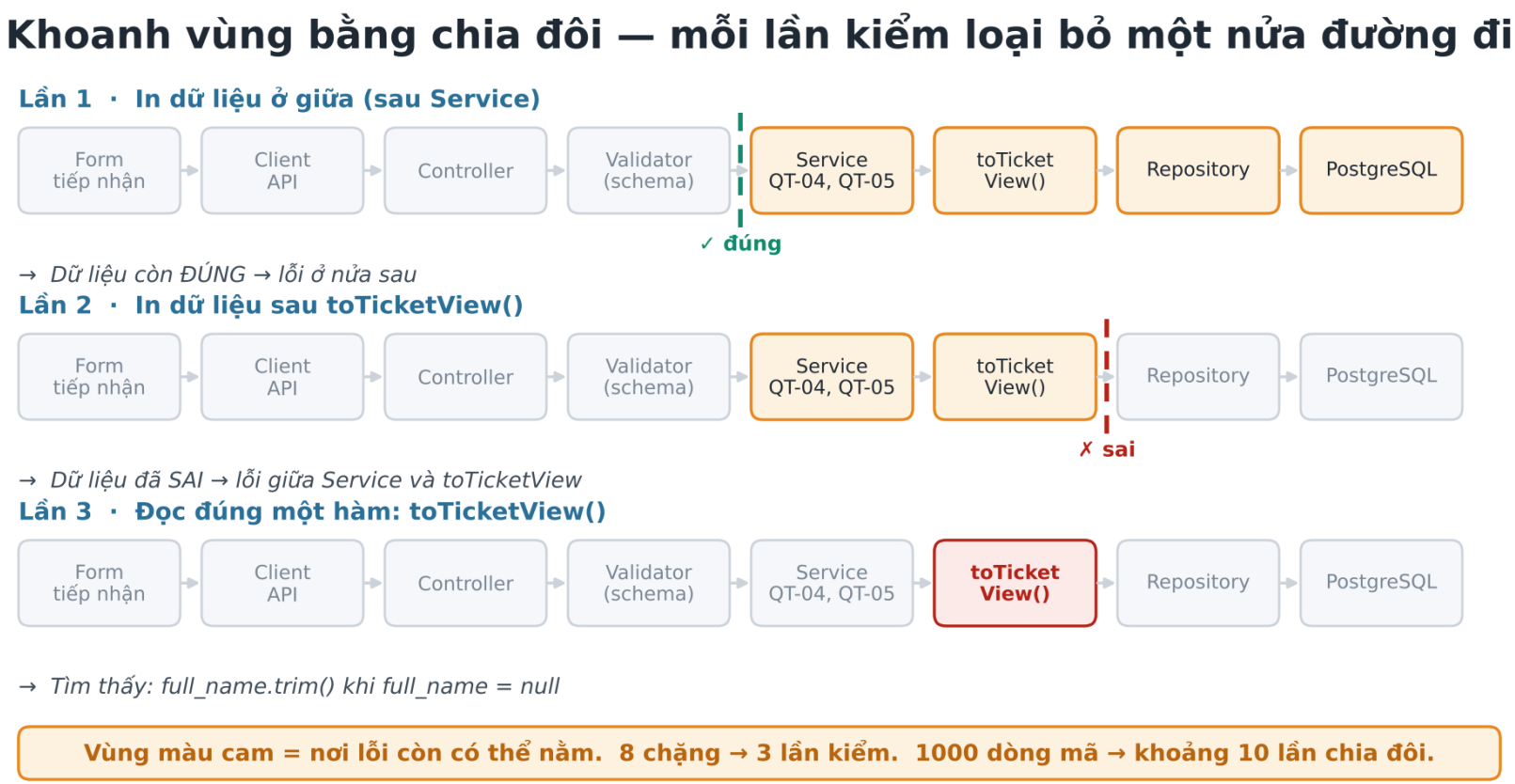
KỸ THUẬT CHO TỪNG BƯỚC

Mỗi bước làm bằng gì, và dấu hiệu em đang làm sai

Bước	Kỹ thuật cụ thể	Công cụ	Dấu hiệu đang làm sai
1. Tái lập	Chạy lại 3–5 lần với cùng đầu vào; **bớt dần từng trường** cho tới khi lỗi biến mất — chỗ vừa bớt là manh mối. Ghi lại lệnh tái lập.	curl, Postman, một test đang FAIL, một ô notebook	"Lúc được lúc không" mà vẫn bắt tay vào sửa
2. Khoanh vùng	**Chia đôi**: in giá trị ở giữa đường đi; đúng thì lỗi ở nửa sau, sai thì ở nửa trước. Với lỗi sau một commit: **git bisect**.	console.log / print có nhãn, debugger, breakpoint(), git bisect	Đọc mã từ dòng đầu tới dòng cuối; sửa thử nhiều chỗ cùng lúc
3. Sửa	Hỏi "vì sao" cho tới khi gặp chỗ **vi phạm hợp đồng** đầu tiên; sửa ở đó. Mỗi lần sửa một thay đổi.	Hợp đồng dữ liệu (buổi 9), schema, quy tắc QT-xx	Thêm if/try ở chỗ lỗi hiện ra; "sửa" bằng cách tắt cảnh báo
4. Xác nhận	Biến lệnh tái lập ở bước 1 thành **một test tự động**; chạy toàn bộ bộ test.	Jest/Vitest, pytest, JUnit	Chỉ thử bằng tay một lần rồi commit "fix bug"

Mẹo mạnh nhất ở bước 1: đầu vào càng nhỏ thì bước 2 càng nhanh. Một lỗi tái lập được bằng 3 trường dữ liệu dễ hơn trăm lần một lỗi chỉ xuất hiện "khi chạy cả hệ thống".

Không đọc từ đầu tới cuối — kiểm ở giữa



Đọc hình thế nào

- Mỗi hàng là một lần kiểm: in dữ liệu tại vạch đứt, so với hợp đồng.
- Đúng (xanh) → lỗi nằm phía sau vạch. Sai (đỏ) → lỗi nằm phía trước.
- Vùng cam thu hẹp một nửa sau mỗi lần: 8 chặng chỉ cần 3 lần kiểm.
- Điểm kiểm tốt nhất là RANH GIỚI giữa các lớp — đúng chỗ hợp đồng dữ liệu đã viết ở buổi 9.
- [DA] Tương tự: đếm số dòng và in df.head() sau mỗi pha. [AI] kiểm shape và vài giá trị sau build_features().

Ví dụ: endpoint trả về 500 khi tạo phiếu

Nhật ký gỡ lỗi — chính là nội dung một mục trong Bug Log của BT3

```
TRIỆU CHỨNG : POST /api/tickets trả về 500, giao diện hiện "Có lỗi xảy ra".

BƯỚC 1 — TÁI LẬP
  Thử 5 lần với cùng dữ liệu: lỗi xuất hiện 5/5 lần → tái lập ổn định.
  Bộ dữ liệu nhỏ nhất gây lỗi:
    { customerId: 1024, deviceId: 3311, issueDesc: "Máy sạc không vào" }
  Bớt từng trường → vẫn lỗi. Đổi customerId khác → HẾT LỖI.
  ⇒ Nghi ngờ: liên quan tới khách hàng có id 1024.

BƯỚC 2 — KHOANH VÙNG (in giá trị ở giữa đường đi)
  controller : dữ liệu vào đúng
  service    : khach = { id: 1024, full_name: null }   ← BẤT THƯỜNG
  repository : câu truy vấn chạy đúng, trả về đúng bản ghi
  ⇒ Lỗi ở toTicketView(), không phải ở repository.

BƯỚC 3 — SỬA (nguyên nhân, không phải triệu chứng)
  Nguyên nhân : toTicketView() gọi khach.full_name.trim() khi full_name = null.
  Gốc rễ      : customers_raw.csv có bản ghi thiếu tên; khi nạp không kiểm.
  Sửa triệu chứng (SAI) : bọc try/catch ở controller rồi trả 200.
  Sửa nguyên nhân (ĐÚNG): hợp đồng ghi rõ tenKhach LUÔN CÓ; hàm chuyển đổi
                          dùng '(không rõ)' khi nguồn rỗng; thêm kiểm chất lượng.

BƯỚC 4 — XÁC NHẬN
  Chạy lại bộ dữ liệu bước 1 → 201 Created. Chạy lại 12 test → 12 PASS.
  Thêm test: "tạo phiếu được khi khách thiếu họ tên".
  Commit: fix(ticket): xử lý khách thiếu họ tên khi tạo phiếu, bổ sung test
```

Ghi nhớ

- Bước 1: bớt dần đầu vào tới khi lỗi biến mất — chỗ vừa bớt là manh mối.
- Bước 2: in giá trị ở RANH GIỚI giữa các lớp, không đọc mã từ đầu.
- Bước 3: tìm "gốc rễ" — vì sao giá trị đó rỗng? Ở đây là dữ liệu nguồn.
- Bước 4: mỗi lỗi sửa xong sinh ra một test mới và một commit có thông điệp rõ.
- Ghi nhật ký này vào docs/bug-log.md NGAY khi gỡ xong.

SỬA TRIỆU CHỨNG HAY SỬA NGUYÊN NHÂN

Cùng một lỗi, hai cách sửa — chỉ một cách làm lỗi biến mất thật

✗ SỬA TRIỆU CHỨNG — lỗi sẽ quay lại ở chỗ khác

```
// controller: "nuốt" lỗi để giao diện không báo đỏ
try {
  const phieu = await ticketService.tao(req.body);
  res.status(201).json({ data: phieu });
} catch (e) {
  res.status(200).json({ data: null });   // ← giả vờ thành công
}

// hoặc: né null ngay tại chỗ lỗi hiện ra
const ten = khach.full_name ? khach.full_name.trim() : '';
// Màn hình danh sách, báo cáo, xuất Excel...
// vẫn gọi full_name.trim() ở 6 chỗ khác.
```

✓ SỬA NGUYÊN NHÂN — sửa một lần, ở ranh giới

```
// 1. Hợp đồng (buổi 9) ghi rõ: tenKhach LUÔN CÓ, không null
function toTicketView(t) {
  return {
    ticketCode: t.ticket_code,
    tenKhach: (t.customer?.full_name ?? '').trim() || '(không rõ)',
    // ... các trường khác
  };
}
// 2. Nguồn: kiểm chất lượng khi nạp customers_raw.csv
//    → dòng thiếu tên vào bảng reject, có lý do
// 3. Test hồi quy: "tạo phiếu được khi khách thiếu họ tên"
```

Hậu quả: lỗi biến mất ở màn hình này nhưng còn ở 6 chỗ khác; người dùng nhận "thành công" nhưng không có phiếu — tệ hơn cả lỗi 500.

Kỹ thuật "5 lần vì sao": vì sao 500? → trim() trên null. Vì sao null? → khách thiếu tên. Vì sao thiếu? → CSV nguồn thiếu, lúc nạp không kiểm. Sửa ở câu trả lời cuối cùng.

BƯỚC 4 → TEST HỒI QUY

Biến lệnh tái lập thành một test chạy mãi mãi

Test hồi quy cho lỗi vừa gỡ — viết bằng Jest [SE] và pytest [DA/AI]

```
// tests/ticket.service.test.js — [SE] Jest
test('tạo phiếu được khi khách thiếu họ tên (bug BUG-004)', async () => {
  // Chuẩn bị: dữ liệu tối thiểu tái lập lỗi ở bước 1
  const repo = taoRepoGia({ khach: { id: 1024, full_name: null } });
  const service = taoTicketService(repo);

  const phieu = await service.tao({ customerId: 1024, deviceId: 3311,
                                    issueDesc: 'Máy sạc không vào' });

  expect(phieu.tenKhach).toBe('(không rõ)');        // hợp đồng được giữ
  expect(phieu.ticketCode).toMatch(/^BH-\d{6}\/\d{4}$/);
});

# tests/test_clean_customers.py — [DA] pytest
def test_dong_thieu_ten_vao_bang_reject():            # bug BUG-004, phía nguồn
    df = pd.DataFrame({"sdt": ["0901234567"], "ho_ten": [None]})
    sach, reject = lam_sach_khach_hang(df)
    assert len(sach) == 0
    assert reject.iloc[0]["ly_do"] == "Thiếu họ tên khách hàng"
```

Ghi nhớ

- Tên test mô tả HÀNH VI mong đợi và ghi mã bug — đọc tên là biết test bảo vệ điều gì.
- Test dùng đúng dữ liệu tối thiểu tìm được ở bước 1.
- Chạy test TRƯỚC khi sửa: phải FAIL. Sau khi sửa: phải PASS. Đó là bằng chứng sửa đúng.
- Test này thuộc loại “dữ liệu sai” — đúng một trong ba loại BT2 yêu cầu (C4).

BUG LOG – GHI NGAY KHI GỠ XONG

Một mục Bug Log đủ bảy dòng (mẫu cho BT3)

Dòng	Nội dung	Ví dụ — lỗi vừa gỡ ở slide demo
1. Mã và mức độ	Mã duy nhất; Critical / Major / Minor	BUG-004 · Major — chức năng tạo phiếu thất bại với một nhóm khách
2. Phát hiện bởi	Test case hoặc tình huống phát hiện	TC-07 “tạo phiếu cho khách nhập từ dữ liệu cũ” (kiểm thử thủ công)
3. Triệu chứng	Người dùng thấy gì	POST /api/tickets trả 500; giao diện hiện “Có lỗi xảy ra”
4. Cách tái lập	Đầu vào tối thiểu + lệnh	curl -X POST … -d '{"customerId":1024,"deviceId":3311,…}'
5. Nguyên nhân	NGUYÊN NHÂN, không phải triệu chứng	toTicketView() gọi trim() trên full_name = null; CSV nguồn thiếu tên, lúc nạp không kiểm
6. Cách sửa	Sửa ở đâu, vì sao ở đó	Hàm chuyển đổi dùng “(không rõ)”; thêm quy tắc kiểm chất lượng; dòng thiếu tên vào reject
7. Commit và xác nhận	Link commit mở được + test	github.com/…/commit/a1b2c3d · test “tạo phiếu được khi khách thiếu họ tên” PASS

Dòng 5 hay bị viết sai nhất: “API trả về 500” là TRIỆU CHỨNG. BT3 yêu cầu ≥ 3 bug, có ít nhất một Critical hoặc Major, và giảng viên sẽ bấm vào từng link commit.

CÔNG CỤ DEBUG THEO TRACK

Đặt điểm kiểm ở đâu và dùng công cụ gì

Track	Điểm kiểm nên đặt	Công cụ	Câu lệnh / thao tác mẫu
SE	Giữa frontend và API (tab Network); đầu controller; trước và sau toXxxView(); câu SQL thực sự chạy	DevTools Network, VS Code debugger, node --inspect, curl -v	curl -v -X POST localhost:8000/api/tickets -H 'Content-Type: application/json' -d @phieu.json
DA	Sau mỗi pha extract / transform / quality / load: số dòng, kiểu cột, giá trị rỗng	breakpoint() / pdb, df.info(), df.shape, SQL COUNT(*)	print(f"[SAU-TRANSFORM] {df.shape} rỗng={df.isna().sum().to_dict()}")
AI	Sau build_features(): shape, thứ tự cột; phân bố nhãn mỗi tập; dự đoán trên 3 mẫu biết trước đáp án	pdb, assert về shape, huấn luyện thử trên 20 mẫu	assert list(X_test.columns) == list(X_train.columns), "lệch thứ tự cột"

Dòng in để gỡ lỗi luôn có NHÃN và phải được xoá (hoặc đổi thành log DEBUG) trước khi commit. Một repo nộp bài còn đầy print("aaa") là dấu hiệu chưa dọn.

03 Logging và xử lý lỗi xuyên lớp

Bốn mức log · ba thứ mọi dòng log phải có · thứ tuyệt đối không ghi · lỗi đi qua các lớp thế nào

Bốn mức log — dùng đúng mức để log có ích

Mức	Dùng khi	Ví dụ trong luồng Smart CRM	Bật ở môi trường thật?
DEBUG	Chi tiết kỹ thuật chỉ hữu ích khi đang phát triển	Giá trị biến trung gian; câu truy vấn được sinh ra	Không — chỉ bật khi cần điều tra
INFO	Sự kiện nghiệp vụ quan trọng đã xảy ra thành công	“Đã tạo phiếu BH-000231/2026 cho khách 1024 bởi nhân viên 12”	Có
WARN	Bất thường nhưng hệ thống vẫn xử lý được	“Khách 1024 thiếu họ tên, dùng giá trị mặc định”; “Bỏ qua 14 dòng không quy đổi được ngày”	Có
ERROR	Sự cố khiến thao tác thất bại, cần người xem	“Không kết nối được CSDL khi tạo phiếu” — kèm toàn bộ ngăn xếp lỗi	Có — và cần được theo dõi

Lỗi NGƯỜI DÙNG (nhập thiếu trường, thiết bị hết bảo hành) KHÔNG ghi ở mức ERROR — đó là nghiệp vụ bình thường. Ghi INFO nếu cần thống kê, hoặc không ghi. Ghi nhầm mức sẽ làm ngập log ERROR và che mất sự cố thật.

Viết log hữu ích và log an toàn

BA THỨ MỌI DÒNG LOG PHẢI CÓ

- **THỜI ĐIỂM** — dấu thời gian có múi giờ, để đối chiếu với báo cáo của người dùng.
- **NGỮ CẢNH** — định danh đối tượng liên quan: mã phiếu, id khách, id người thực hiện, **requestId**. Không có ngữ cảnh thì không truy được về trường hợp cụ thể.
- **CHUYỆN GÌ XẢY RA** — mô tả bằng lời, không chỉ một mã lỗi.
- Log tồi: “Error in createTicket”.
- Log tốt: “2026-10-26T14:30:00+07:00 ERROR taoPhieu thất bại — requestId=7f3a-21, customerId=1024, deviceId=3311, nguyên nhân: mất kết nối CSDL khi tạo phiếu”.

TUYỆT ĐỐI KHÔNG GHI VÀO LOG

- Mật khẩu, khoá API, token, chuỗi kết nối đầy đủ — kể cả ở mức DEBUG, kể cả “chỉ tạm thời”.
- Số điện thoại, địa chỉ đầy đủ của khách — QT-15 yêu cầu che trên giao diện thì log cũng phải tuân thủ.
- Toàn bộ thân request/response chứa dữ liệu cá nhân.
- **Lý do:** log được lưu lâu hơn dữ liệu, sao chép sang nhiều nơi, nhiều người đọc, và ít khi được bảo vệ như CSDL.
- **Nguyên tắc thực dụng:** ghi ID thay vì ghi dữ liệu — “customerId=1024”, không phải tên và số điện thoại.

VÍ DỤ — CẤU HÌNH LOG MỘT LẦN, DÙNG MỌI NƠI

Log có cấu trúc trong Node.js và Python

Ghi ra stdout, mỗi dòng một sự kiện, có mức và ngữ cảnh

```
// ===== [SE] Node.js — dùng pino (hoặc logger tự viết ở tài liệu tự học A3)
const logger = require('pino')({ level: process.env.LOG_LEVEL || 'info',
  redact: ['req.headers.authorization', '*.password', '*.soDienThoai'] });

logger.info({ requestId, ticketCode: 'BH-000231/2026', customerId: 1024 },
            'đã tạo phiếu');
logger.error({ requestId, err }, 'tạo phiếu thất bại: không kết nối được CSDL');
// → {"level":30,"time":1793...,"requestId":"7f3a-21","ticketCode":"BH-000231/2026",...}

# ===== [DA/AI] Python — cấu hình MỘT lần ở điểm vào chương trình
import logging, os
logging.basicConfig(
    level=os.getenv("LOG_LEVEL", "INFO"),
    format="%(asctime)s %(levelname)s %(name)s [lo=%(ma_lo)s] %(message)s",
    defaults={"ma_lo": "-"},              # Python ≥ 3.10
)
log = logging.getLogger("pipeline.transform")

log.warning("bỏ qua %d dòng không quy đổi được ngày", so_dong_loi,
            extra={"ma_lo": ma_lo})
try:
    nap_kho(df)
except Exception:
    log.exception("nạp kho thất bại")     # tự kèm ngăn xếp lỗi, mức ERROR
    raise
```

Ghi nhớ

- Cấu hình log MỘT lần ở điểm vào; các module chỉ lấy logger theo tên.
- Mức log lấy từ biến môi trường LOG_LEVEL — không sửa mã để bật DEBUG.
- **redact** (pino) che trường nhạy cảm tự động, kể cả khi em quên.
- **log.exception** = mức ERROR + ngăn xếp lỗi. Không dùng print trong mã nộp bài.
- Ghi ra stdout, không ghi tệp trong container — docker compose logs đọc được.

Mỗi lớp một trách nhiệm khi có lỗi

1

Repository / lớp dữ liệu — DỊCH lỗi kỹ thuật

Bắt lỗi của thư viện CSDL (ECONNREFUSED, vi phạm khoá duy nhất), bọc thành lỗi của ứng dụng: **LoiTruyCapDuLieu**, **LoiTrungKhoa**. Không trả mã HTTP, không ghi log ở đây.

2

Service / lớp nghiệp vụ — NÉM lỗi nghiệp vụ có mã

Kiểm quy tắc QT-xx; vi phạm thì ném **LoiNghiepVu('PHIEU_DANG_MO', thông điệp)**. Không bắt lỗi hạ tầng mà mình không xử lý được — để nó đi lên.

3

Controller + middleware lỗi — CHUYỂN thành phản hồi, GHI LOG một lần

Một middleware duy nhất ánh xạ loại lỗi → mã HTTP và định dạng { error, message, fields } của buổi 9; ghi log kèm requestId; ẩn chi tiết kỹ thuật với người dùng.

4

Giao diện / người dùng — HIỂN THỊ có hướng dẫn

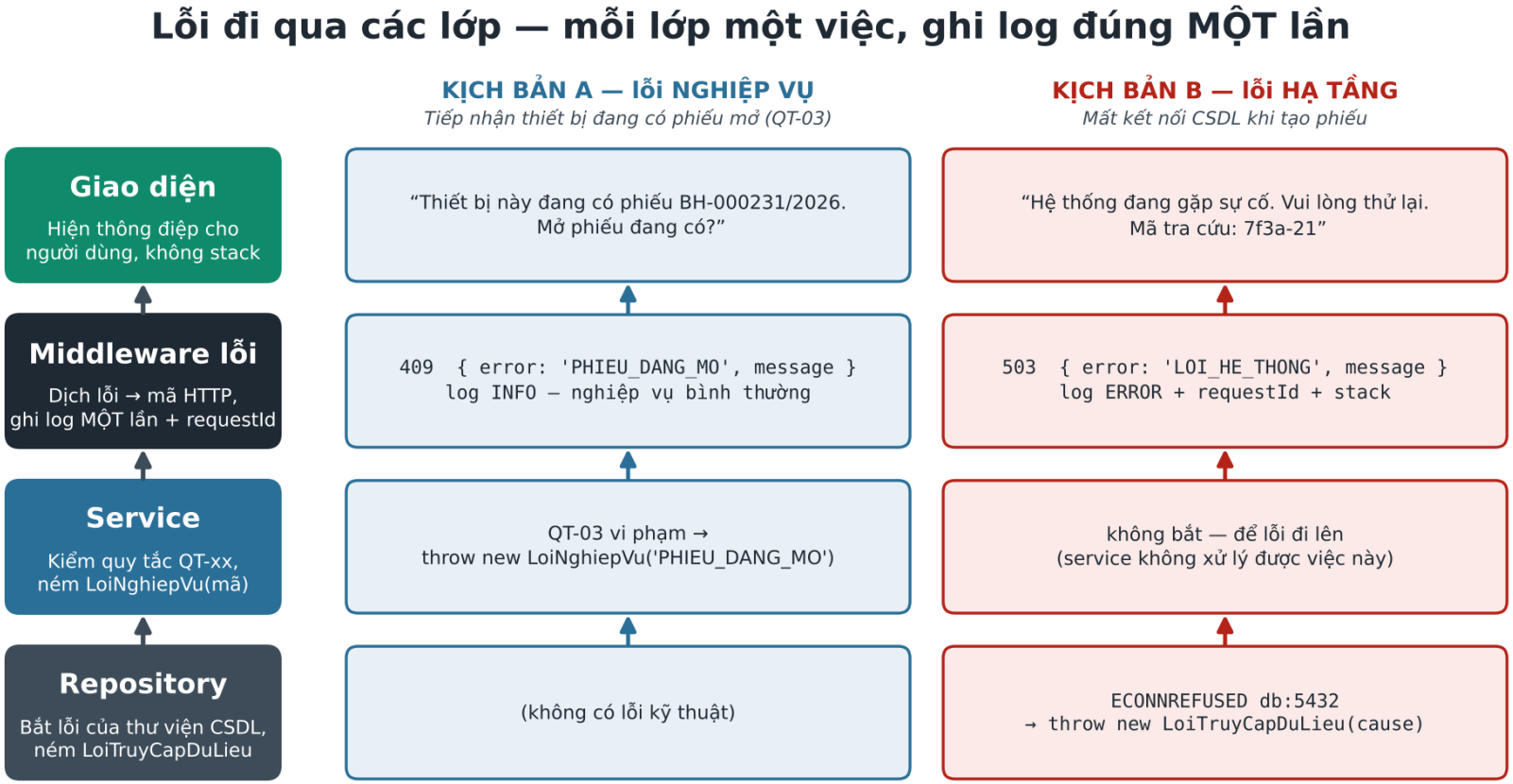
Rẽ nhánh theo **error** (mã máy đọc), hiện **message** (câu người đọc): chuyện gì · vì sao · làm gì tiếp. 5xx thì kèm mã tra cứu để người dùng báo lại.

Ba quy tắc vàng

- **Bắt ở nơi xử lý được.** Không xử lý được thì đừng bắt.
- **Ghi log đúng một lần**, ở lớp trên cùng. Log ở cả ba lớp = một lỗi thành ba dòng ERROR.
- **Dịch ở ranh giới.** Lớp nghiệp vụ không biết HTTP; lớp dữ liệu không biết người dùng.

MINH HOẠ — LỖI ĐI QUA CÁC LỚP

Hai loại lỗi, hai đường đi khác nhau



Đọc hình thế nào

- Đọc từ dưới lên: lỗi sinh ra ở lớp dưới, đi lên tới người dùng.
- Kịch bản A: vi phạm quy tắc QT-03 → 409, log INFO. Người dùng được chỉ lối đi tiếp.
- Kịch bản B: CSDL mất kết nối → service KHÔNG bắt → middleware trả 503, log ERROR kèm requestId.
- Người dùng chỉ thấy mã tra cứu; kỹ sư dùng mã đó tìm đúng dòng log.
- Không lớp nào trả về 200 khi có lỗi.

Express: ném ở service, dịch và ghi log ở một chỗ

src/errors.js và src/middleware/xuLyLoi.js — [SE]; FastAPI tương tự bằng exception_handler

```
// ---------- src/errors.js — các loại lỗi của ứng dụng ----------
class LoiNghiepVu extends Error {            // vi phạm quy tắc QT-xx
  constructor(ma, thongDiep, httpStatus = 409) { super(thongDiep);
    this.ma = ma; this.httpStatus = httpStatus; }
}
class LoiTruyCapDuLieu extends Error {       // CSDL hỏng, mất kết nối
  constructor(cause) { super('Lỗi truy cập dữ liệu', { cause }); }
}

// ---------- service: KHÔNG biết HTTP ----------
if (await repo.coPhieuDangMo(deviceId))
  throw new LoiNghiepVu('PHIEU_DANG_MO', 'Thiết bị đang có phiếu chưa hoàn tất (QT-03)');

// ---------- middleware lỗi: đăng ký CUỐI CÙNG trong app.js ----------
function xuLyLoi(err, req, res, next) {
  if (err instanceof LoiNghiepVu) {
    logger.info({ requestId: req.requestId, ma: err.ma }, 'vi phạm quy tắc nghiệp vụ');
    return res.status(err.httpStatus).json({ error: err.ma, message: err.message });
  }
  logger.error({ requestId: req.requestId, err }, 'lỗi hệ thống');   // log MỘT lần
  res.status(err instanceof LoiTruyCapDuLieu ? 503 : 500).json({
    error: 'LOI_HE_THONG',
    message: `Hệ thống đang gặp sự cố. Vui lòng thử lại. Mã tra cứu: ${req.requestId}`,
  });                                        // KHÔNG gửi err.stack cho người dùng
}
app.use(xuLyLoi);
```

Ghi nhớ

- Service ném lỗi có MÃ, controller không cần try/catch ở từng route.
- Middleware là nơi DUY NHẤT biết ánh xạ lỗi → mã HTTP.
- Lỗi nghiệp vụ: INFO. Lỗi hệ thống: ERROR kèm ngăn xếp — chỉ trong log.
- Người dùng nhận mã tra cứu = requestId.
- Express 4: route async phải chuyển lỗi bằng next(err) (hoặc dùng express-async-errors); Express 5 tự chuyển.

NĂM KIỂU XỬ LÝ LỖI SAI HAY GẶP

Nhận ra trong repo của mình và sửa trước buổi 12

Kiểu sai	Trông như thế nào	Hậu quả	Sửa thành
Nuốt lỗi	catch (e) {} · except: pass	Lỗi biến mất không dấu vết; dữ liệu sai lặng lẽ	Bắt đúng loại lỗi xử lý được; còn lại để đi lên
Giả vờ thành công	catch → res.status(200) · trả về null, []	Người dùng tưởng đã lưu; lỗi hạ tầng bị đổi nghĩa thành “không có dữ liệu”	Trả đúng mã 4xx/5xx theo định dạng lỗi buổi 9
Log ở mọi lớp	Repository, service, controller cùng log rồi ném lại	Một sự cố thành 3 dòng ERROR; đếm sự cố sai	Chỉ middleware / điểm vào ghi log
Lộ chi tiết kỹ thuật	Trả err.stack, câu SQL, tên bảng cho người dùng	Lộ cấu trúc hệ thống; người dùng không hiểu	Thông điệp chung + mã tra cứu; chi tiết vào log
Thử lại vô hạn, không timeout	while (true) gọi lại dịch vụ; fetch không có thời gian chờ	Một dịch vụ treo kéo cả hệ thống treo	Timeout + thử lại có giới hạn (≤ 3 lần, có giãn cách)

*Phép thử nhanh: grep -rnE "catch *\([^)]*\) *\{ *\}|except *: *pass" src/ — kết quả phải trống.*

04 Docker cơ bản – ở mức chạy được

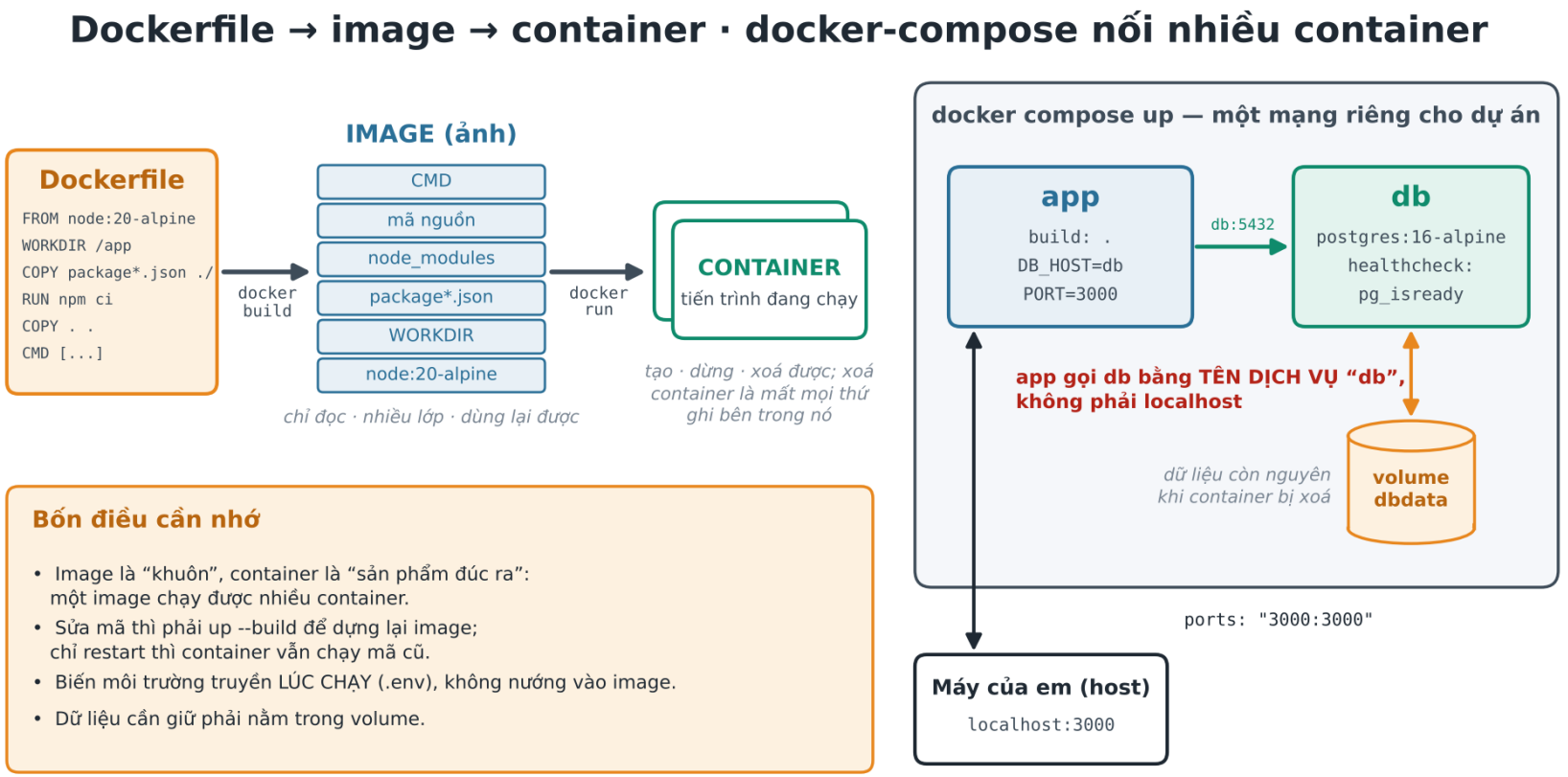
Vì sao cần container · image, container, volume · Dockerfile tối thiểu · docker-compose cho ứng dụng kèm CSDL

VÌ SAO HỌC PHẦN NÀY DÙNG DOCKER

Docker biến câu “trên máy em chạy được” thành “trên máy nào cũng chạy được”.

- Một container đóng gói cùng nhau: mã nguồn, **runtime đúng phiên bản**, thư viện đúng phiên bản, cấu hình hệ điều hành. Người khác chỉ cần Docker — không cần cài Node, Python hay PostgreSQL.
- Đây là nguyên tắc **TÁI LẬP ĐƯỢC** của học phần, hiện thực bằng công cụ cụ thể: cả năm khác biệt ở phần 1 được chặn một lần.
- **BT2 KHÔNG bắt buộc Docker** (có thì được đánh giá cao ở tiêu chí 2). **BT3 bắt buộc** gói chạy được bằng ≤ 2 lệnh từ máy chưa cài gì. Máy yếu không chạy được Docker: dùng script cài đặt (setup.sh / Makefile) ≤ 2 lệnh.
- Phạm vi học phần: chỉ ĐÓNG GÓI để người khác dựng lại được. Triển khai lên môi trường thật, CI/CD và vận hành thuộc **Chuyên đề tốt nghiệp 2**.

Image, container, volume và mạng của compose



Bốn điều cần nhớ

- Image là "khuôn", container là "sản phẩm đúc ra": một image chạy được nhiều container.
- Sửa mã thì phải up --build để dựng lại image; chỉ restart thì container vẫn chạy mã cũ.
- Biến môi trường truyền LÚC CHẠY (.env), không nướng vào image.
- Dữ liệu cần giữ phải nằm trong volume.

Đọc hình thế nào

- Dockerfile là công thức; docker build tạo ra IMAGE gồm nhiều lớp chỉ đọc.
- docker run tạo CONTAINER từ image — một tiến trình đang chạy, xoá được.
- Compose tạo một mạng riêng: các dịch vụ gọi nhau bằng TÊN (db), không bằng localhost.
- VOLUME giữ dữ liệu CSDL khi container bị xoá.
- ports đưa một cổng của container ra máy em để mở trình duyệt.

Dockerfile cho ứng dụng Node và cho script Python

Chọn MỘT trong hai theo track của em

```
# ===== Dockerfile — track SE (Node.js) =====
FROM node:20-alpine
WORKDIR /app
# Chép tệp khoá TRƯỚC rồi cài — Docker cache lớp này,
# lần build sau không phải cài lại nếu thư viện không đổi
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 3000
CMD ["node", "src/index.js"]

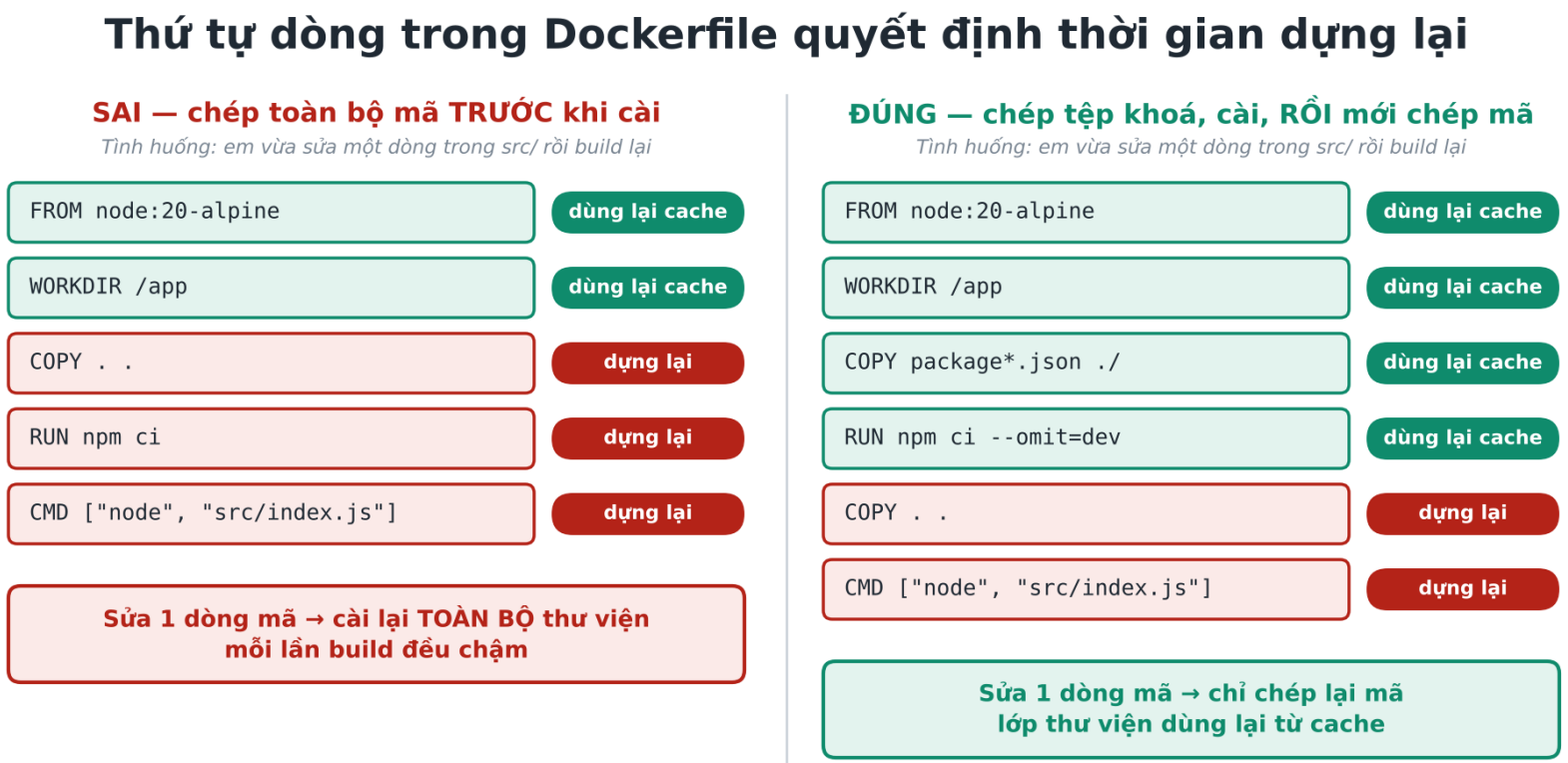
# ===== Dockerfile — track DA / AI (Python) =====
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "-m", "src.pipeline"]

# ===== .dockerignore — BẮT BUỘC, đặt cạnh Dockerfile =====
node_modules
.venv
__pycache__
.git
.env
data/raw
```

Ghi nhớ

- Thứ tự dòng có ý nghĩa: chép tệp khoá → cài → rồi mới chép mã (slide sau).
- Luôn ghim phiên bản ảnh nền: node:20-alpine, không dùng node:latest.
- .dockerignore là BẮT BUỘC. Thiếu nó, image chứa cả node_modules của máy em và tệp .env thật.
- Chép .env vào image là lỗi bảo mật nghiêm trọng — biến môi trường truyền lúc chạy.
- BT3 sẽ kiểm: từ máy chưa cài gì, ≤ 2 lệnh là hệ thống lên được.

Vì sao chép tệp khoá trước rồi mới chép mã



Đọc hình thế nào

- Mỗi dòng Dockerfile tạo một LỚP; Docker dùng lại lớp nếu đầu vào của nó không đổi.
- Một lớp thay đổi → mọi lớp phía dưới phải dựng lại.
- Bên trái: sửa mã làm COPY . . đổi → npm ci chạy lại mỗi lần.
- Bên phải: tệp khoá không đổi → lớp cài thư viện được dùng lại; chỉ chép lại mã.
- Áp dụng y hệt cho Python: COPY requirements.txt → pip install → COPY . .

docker-compose.yml: đúng thứ BT3 sẽ chấm

docker compose up → cả ứng dụng và cơ sở dữ liệu cùng lên

```
services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: smartcrm
      POSTGRES_USER: ${DB_USER}            # lấy từ .env, KHÔNG ghi cứng
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - dbdata:/var/lib/postgresql/data    # giữ dữ liệu khi container tắt
      - ./migrations:/docker-entrypoint-initdb.d   # tự chạy SQL khi khởi tạo
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER} -d smartcrm"]
      interval: 5s
      retries: 10
  app:
    build: .
    environment:
      DB_HOST: db                          # TÊN DỊCH VỤ, không phải localhost
      DB_USER: ${DB_USER}
      DB_PASSWORD: ${DB_PASSWORD}
    ports:
      - "3000:3000"
    depends_on:
      db: { condition: service_healthy }   # chờ db SẴN SÀNG mới khởi động app
volumes: { dbdata: }
```

Ghi nhớ

- DB_HOST là “db” — tên dịch vụ, không phải localhost. Lỗi số một khi chạy trong container.
- healthcheck + condition: service_healthy: app chờ CSDL sẵn sàng, không chỉ chờ container khởi động.
- Thư mục migrations chỉ tự chạy khi volume còn TRỐNG. Sửa SQL rồi thì: docker compose down -v.
- Mật khẩu lấy từ .env; tệp compose được commit nên không ghi cứng.
- Hai lệnh BT3: cp .env.example .env rồi docker compose up --build.

BẢY LỆNH DÙNG HẰNG NGÀY

Đủ để dựng, xem, gỡ lỗi và dọn hệ thống

Lệnh	Làm gì	Dùng khi
docker compose up -d --build	Dựng lại image nếu mã đổi, khởi động mọi dịch vụ ở nền	Mỗi lần sửa mã và muốn chạy lại
docker compose ps	Liệt kê dịch vụ và trạng thái (running / healthy / exited)	Kiểm tra mọi thứ đã lên; db phải là healthy
docker compose logs -f app --tail 50	Xem log của một dịch vụ, theo dõi liên tục	Bước đầu tiên khi có lỗi — ĐỌC LOG TRƯỚC
docker compose exec db sh -c 'psql -U "$POSTGRES_USER" -d "$POSTGRES_DB"'	Mở psql bên trong container đang chạy; biến được đọc TRONG container	Kiểm dữ liệu đã nạp: SELECT COUNT(*) …
docker compose run --rm pipeline	Chạy một dịch vụ một lần rồi xoá container	[DA] chạy pipeline; [AI] chạy huấn luyện
docker compose down	Dừng và xoá container, GIỮ volume dữ liệu	Kết thúc buổi làm việc
docker compose down -v	Xoá cả volume — CSDL trở về trống	Sửa migration; làm phép thử từ con số không

Biến trong .env KHÔNG có sẵn trong shell của máy em — vì thế lệnh psql dùng sh -c '…' để đọc biến bên trong container. Docker Compose v2 dùng "docker compose" (có dấu cách); "docker-compose" là bản cũ.

LỖI DOCKER HAY GẶP

Đọc triệu chứng — biết sửa ở đâu

Triệu chứng	Nguyên nhân thường gặp	Cách sửa
ECONNREFUSED 127.0.0.1:5432 / connection refused	App gọi localhost — trong container, localhost là chính nó	DB_HOST=db (tên dịch vụ trong compose)
App thoát ngay khi khởi động, chạy lại thì được	App khởi động trước khi CSDL sẵn sàng	healthcheck + condition: service_healthy; thêm vòng thử kết nối
Bảng không tồn tại dù đã có tệp SQL	Volume đã có dữ liệu cũ nên initdb không chạy lại	docker compose down -v rồi up lại
Sửa mã mà hệ thống vẫn chạy như cũ	Chỉ restart, không dựng lại image	docker compose up -d --build
Bind for 0.0.0.0:3000 failed: port is already allocated	Cổng đã bị tiến trình khác chiếm	Đổi cổng qua biến: "${API_PORT:-3000}:3000"
Image nặng hàng GB, build rất chậm	Thiếu .dockerignore; ảnh nền đầy đủ thay vì slim/alpine	Thêm .dockerignore; dùng -slim / -alpine
exec ./start.sh: no such file or directory (tệp có thật)	Tệp .sh lưu với xuống dòng Windows (CRLF)	Lưu tệp ở LF; thêm .gitattributes: *.sh text eol=lf

Quy trình debug bốn bước vẫn áp dụng: docker compose logs là bước tái lập và khoanh vùng; đừng sửa Dockerfile khi chưa đọc log.

05

Phần chuyên ngành – tự học theo tài liệu

Đóng gói và cấu hình theo stack [SE] · nối pipeline → kho → dashboard [DA] · đóng gói mô hình thành dịch vụ dự đoán [AI]

PHẦN CHUYÊN NGÀNH – SINH VIÊN TỰ HỌC THEO TÀI LIỆU

Việc phải làm trước workshop buổi 12 theo chuyên ngành

Tài liệu tự học buổi 11 (song ngữ Việt – Anh) có tệp cấu hình hoàn chỉnh sao chép được và phép thử tự kiểm cho từng track. Phần D dùng chung: Bug Log, logging, xử lý lỗi.

SE — Ứng dụng web nhiều tầng

- Dockerfile nhiều giai đoạn, chạy bằng người dùng không đặc quyền, có HEALTHCHECK
- Compose ba dịch vụ db · api · web, thứ tự khởi động bằng service_healthy
- Logger JSON ra stdout, có requestId, che trường nhạy cảm
- Middleware lỗi duy nhất theo định dạng buổi 9
- Chạy phép thử năm bước phần A4

DA — Pipeline → kho → dashboard

- Compose warehouse · pipeline · dashboard; dữ liệu nguồn gắn volume chỉ đọc
- Điều phối ba pha bằng một lệnh; kiểm chất lượng TRƯỚC khi nạp
- Báo cáo lần chạy luôn được ghi, kể cả khi thất bại; mã thoát 0/1/2
- Hẹn lịch mức 1–2; chạy bù một ngày
- Chạy phép thử sáu bước phần B4

AI — Mô hình thành dịch vụ dự đoán

- FastAPI: nạp mô hình MỘT lần, /health, top 3, ngưỡng tin cậy
- Trả phiên bản mô hình trong mọi phản hồi
- Ghim scikit-learn bằng == để mô hình nạp được trong image
- API chính gọi mô hình có timeout và suy giảm có kiểm soát
- Chạy phép thử sáu bước phần C4

TỰ KIỂM TRA NHANH

Năm câu hỏi trước khi rời lớp

1. **Tệp nào người chấm cần để cài đúng phiên bản em đã dùng: package.json hay package-lock.json? Vì sao?**
   Gợi ý: một tệp nói "khoảng cho phép", tệp kia nói "bản đã cài thật".

2. **Vì sao không được sửa lỗi trước khi tái lập được nó ổn định?**
   Gợi ý: sau khi sửa, làm sao em biết lỗi biến mất là nhờ bản sửa?

3. **"API trả về 500" là nguyên nhân hay triệu chứng? Viết lại thành một dòng nguyên nhân đúng.**
   Gợi ý: dùng kỹ thuật hỏi "vì sao" liên tiếp.

4. **Service bắt lỗi mất kết nối CSDL rồi trả về null. Điều gì sai ở đây và lỗi nên được xử lý ở lớp nào?**
   Gợi ý: controller sẽ hiểu null là gì? người dùng sẽ thấy gì?

5. **Trong docker-compose, vì sao app không kết nối được CSDL qua localhost, và depends_on đơn thuần thiếu gì?**
   Gợi ý: localhost trong container là ai? container khởi động khác CSDL sẵn sàng thế nào?

NỘI DUNG NỘP BÀI TẬP 2 · 20% ĐIỂM HỌC PHẦN

Nộp gì cho Bài tập 2 — và nộp thế nào

1 · Hình thức và thời hạn

- Nộp qua **VLU E-learning**: **1 file PDF + link GitHub repo có tag v0.2**.
- Tên file: **BT2_<MSSV>_<Họ tên không dấu>.pdf** — một file duy nhất, không nộp .docx.
- Hạn: **trước khi kết thúc buổi 12**.
- Nộp bù: **tối đa 3 ngày**, trừ **10% điểm mỗi ngày** trễ.
- Quá 3 ngày: **0 điểm** thành phần và **không đủ điều kiện dự thi** cuối kỳ.
- Repo để **PUBLIC** — giảng viên không nhận lời mời collaborator. Vì vậy repo tuyệt đối không chứa .env hay secret.

2 · Năm thành phần trong repo (tag v0.2)

- **TP1 – Module cốt lõi** chạy được, tách lớp. [SE] ≥ 2 thực thể đủ CRUD · [DA] pipeline 3 pha, ≥ 5 hàm kiểm chất lượng · [AI] script huấn luyện + mô hình cơ sở.
- **TP2 – ≥ 3 kiểm thử** thuộc ba loại (luồng đúng · biên · dữ liệu sai), PASS bằng một lệnh.
- **TP3 – Module mở rộng** và phần trình bày kết quả, đủ ba trạng thái.
- **TP4 – Hệ thống chạy được** theo README: tệp khoá, .env.example, dữ liệu mẫu.
- **TP5 – Tài liệu:** README.md · docs/thay-doi-thiet-ke.md · docs/ai-disclosure.md.

3 · Cấu trúc file PDF (6–9 trang)

- **Trang bìa:** link repo + tag v0.2, track, mã luồng nghiệp vụ, họ tên – MSSV – lớp.
- **Mục 1** Tóm tắt hệ thống · **Mục 2** Kiến trúc đã hiện thực.
- **Mục 3** Module cốt lõi — **ảnh chụp** chạy được.
- **Mục 4** Bộ kiểm thử — **ảnh chụp** kết quả PASS.
- **Mục 5** Trình bày kết quả — **ảnh chụp** đủ ba trạng thái.
- **Mục 6** Cách dựng lại hệ thống · **Mục 7** Thay đổi so với BT1.
- **Phụ lục (bắt buộc):** bảng khai báo sử dụng công cụ AI.

ẢNH CHỤP KẾT QUẢ CHẠY THẬT là minh chứng bắt buộc cho mục 3, 4, 5 của PDF: thấy rõ lệnh đã chạy và kết quả (terminal, giao diện, test PASS). Mục nào không có ảnh chụp thì tính là **không có minh chứng** — chụp ngay khi chạy được, đừng để tới lúc nộp.

KẾT THÚC BUỔI HỌC

Chốt buổi 11 và chuẩn bị cho buổi 12 — ★ NỘP BÀI TẬP 2

Tự học, tự nghiên cứu

- Đọc tài liệu tự học buổi 11 theo track và phần D chung; chạy PHÉP THỬ TỰ KIỂM của track mình.
- Commit tệp khoá phiên bản; ghi phiên bản runtime vào README mục 2.
- Tạo docs/bug-log.md; ghi lại mọi lỗi đã gỡ từ buổi 8 theo mẫu 7 dòng.
- Rà grep: catch rỗng, console.log/print, dữ liệu nhạy cảm trong log, đường dẫn tuyệt đối.
- Bổ sung test cho tới khi đủ ≥ 3 test thuộc ba loại, PASS bằng một lệnh.
- Chụp ảnh kết quả chạy NGAY khi chạy được (module cốt lõi, test PASS, ba trạng thái) — minh chứng bắt buộc trong PDF.
- THỬ clone repo vào thư mục mới và làm đúng README — bắt buộc trước buổi 12.
- (Không bắt buộc) Viết Dockerfile + compose và chạy thử để làm quen trước buổi 15.

Buổi 12 (thực hành) — Workshop tích hợp · ★ NỘP BT2

- Hoàn thiện tích hợp: luồng chính chạy end-to-end, không lỗi treo.
- Dọn repo theo bảy việc; tạo và đẩy tag v0.2.
- Demo nhanh cá nhân trước lớp.
- Giảng viên CLONE REPO VÀ CHẠY THỬ theo README ngay tại lớp; hỏi bộ ba câu hỏi chuẩn về một hàm bất kỳ.
- ★ NỘP BÀI TẬP 2 qua VLU E-learning: 1 file PDF + link GitHub repo có tag v0.2 — 20% điểm học phần.